CheckInOut – Hostel Management System

CS 432 – Databases (Course Project – Assignment 3)

Module B: Multi-User Behaviour and Stress Testing Report

Semester II (2025–2026)

Date: 28 March 2026

Instructor: Dr. Yogesh K. Meena

Team Members & Contributions

Vipul Sunil Patil – Race condition mitigation & dashboard logic
Sai – Backend transaction handling & WAL optimization
Daksh Jain – Stress testing suite & data collection
Daksh Dave – Failure simulation & report writing
Harshit – Verification tooling & documentation

GitHub Repository:
https://github.com/vipulSP2108/CheckInOut-Hostel-Management-System

Video Demonstration:
https://www.youtube.com/watch?v=Xnf_NpJOGJw

© Indian Institute of Technology Gandhinagar

CheckInOut: Multi-User Behaviour and Stress Testing Report

Assignment 3 Team
Indian Institute of Technology Gandhinagar
Gandhinagar, India
hostel.admin@iitgn.ac.in

Abstract

This report documents the architectural improvements for the CheckInOut Hostel Management System, focusing on four pillars: (1) **Concurrency Control** via SQLite WAL; (2) **Race Condition Mitigation** through `BEGIN IMMEDIATE` transactions; (3) **Stress Testing (write and read)** under high-velocity parallel loads (same time 200+ req); and (4) **Failure Simulation** for ACID verification. Experimental results confirm zero data corruption and 100% transaction integrity under extreme load.

Keywords

Concurrency Control, Race Conditions, SQLite WAL, ACID Compliance, Stress Testing, Atomic Transactions, Hostel Management

1 Introduction & Operating Principles

In this Assignment, we transitioned CheckInOut from a prototype to a robust concurrent platform. Our implementation was guided by the **ACID** principles:

- **Atomicity**: Operations (e.g., room allocation) must fully complete or fully roll back across tables.
- **Consistency**: Constraints (e.g., `Occupancy` ≤ `Capacity`) must be preserved despite parallel load.
- **Isolation**: Parallel scans or dashboard reads must not interfere with each other.
- **Durability**: Committed data must persist through system failures or crashes.

To ensure our analysis is both robust and realistic, we have adopted/assumed **IIT Gandhinagar (IITGN)** (approx. 2,500 students) as our primary deployment model. This institutional context allows us to analyze the system against deterministic "rush periods" (e.g., academic transitions and morning dashboard surges), justifying our implementations, atomic transactions, and a multi-channel stress testing suite.

2 System Analysis & Design Assumptions

Effective stress testing must be grounded in real-world usage patterns. We analyze the hostel domain using **IIT Gandhinagar (IITGN)** (approximately 2,500 students) as our deployment model to identify critical bottleneck points.

2.1 Deployment Model: The IITGN Context

A campus of this scale experiences deterministic "Rush Periods":

- **User Login & Dashboard Fetch**: Every time a student opens the app, the first request triggered is to fetch their profile.
- **Admission Rush**: Simultaneous room allocations and profiles for 1,000+ new students.
- **Resumed Semester Gate Rush**: Massive spikes (1,000 students+) during academic transitions.
- **Write (Complaint/allocations) Spikes**: At the start of each semester, there is a massive concurrent allocations (room booking) and complaints (initial maintenance reporting).

These real-world scenarios assumptions at IIT Gandhinagar formed the foundation for our architectural choices (WAL/Immediate)

2.2 Endpoint Priority & Rationale

System operations are prioritized based on their impact on student life and database integrity.

Table 1: Comprehensive Endpoint Priority Matrix

Priority	Type + Endpoint	Rationale
CRITICAL	**Read** – `GET /api/members/:id`	**User Experience**: First hit immediately after login; critical for app responsiveness.
CRITICAL	**Write** – `POST /api/allocations`	**High-Stakes Integrity**: Must prevent double-room assignment during admission.
HIGHEST	**Write** – `POST /api/scans/gate`	**Throughput**: High-frequency I/O foundation for all security tracking.
HIGH	**Write** – `POST /api/complaints`	**Write Foundation**: action triggered by thousands during semester start.
HIGH	**Write** – `POST /api/auth/login`	**CPU-Intensive**: Session start; must interleave with gate writes without blocking.
MEDIUM	**Read** – `GET /api/maintenance`	**Admin Dashboard**: Real-time triage for hostel wardens.
MEDIUM	**Write** – `POST /api/visitors`	**Security Logging**: Low frequency tracking of guest entry/exit.
MEDIUM	**Read** – `GET /api/furniture`	**Inventory Audit**: Occasional checks of dormitory assets.
MEDIUM	**Read** – `GET /api/fees/member/:id`	**Financial Inquiry**: High volume during fee deadlines.

2.3 Scaling Math: Write Time vs. Capacity

Modeling an extreme "Rush" (e.g., during a semester start):

- **Theoretical Capacity**: Each QR scan write takes ~1ms. of **100 scans/sec** consume only 100ms of database time per 1,000ms.

,
2026.

- **Write Interleaving**: Even if **50 concurrent students** submit complaints (WRITE) during this rush (taking 2ms each), the total DB time is $100 + (50 \times 2) = 200$ms.

Result: Even under extreme "Admission Rush" load, the database is **80% idle**. Our chosen foundations allow for this massive headroom, ensuring zero blocking.

3 Architectural Foundation & Tooling

Optimizing a high-concurrency system requires decoupling configuration, logic, and testing.

3.1 Core Components

- **server.js / database.js**: Initializes Express and the `sqlite3` connection with **WAL mode** and **busy_timeout = 5000**.
- **src/routes/allocations.js**: Houses the most critical business logic, using `BEGIN IMMEDIATE` for atomic room booking.

3.2 Logging

- `audit.log`: Records all state-modifying actions (POST / PATCH / DELETE) per user.
- `access.log`: Logs every HTTP request including response time.
- `scripts/logs/`: Scenario-specific traces for verification

3.3 Test Setup (`scripts/name-test.cjs`)

The core of our verification strategy is a decoupled Node.js testing framework. This allows researchers and admins to execute heavy loads without being tethered to a UI.

- **Decoupled Execution**: Tests are triggered via CLI, making them suitable. The **Worker Pool Runner** implements a sliding-window worker pool, ensuring that exactly N requests remain in-flight.
- **Centralized Invariants**: To ensure reproducibility across testing phases, all operational parameters and data pools are centralized within `scripts/constants.cjs`. This design enables flexible scenario configuration—for instance, simulating a "Gate Rush" workload—by simply modifying invariants without requiring changes to the core testing logic.

3.3.1 Manual Execution & CLI Workflow To ensure the system works independently, we provided a standardized CLI workflow:

(1) **Scenario Configuration**: Update `scripts/constants.cjs` to define the desired `CONCURRENCY` (e.g., 300) and `TOTAL_REQUESTS` (e.g., 2000).
(2) **Individual Stress**: Execute `node scripts/stress-test.cjs` to saturate a specific endpoint (e.g., `POST /api/scans/gate` with 200 workers).
(3) **Race Condition**: Execute `node scripts/race-test.cjs` To simulate concurrent requests (e.g., 5 members) targeting the same resource (e.g., a single room).
(4) **Write—Heavy Stress (Complaints)**: To simulate high-volume maintenance reporting (e.g., 50 parallel writes) execute `node scripts/stress_test.cjs`.
(5) **Atomicity Verification**: To evaluate system recovery under forced termination (e.g., crashing a student during a `COMMIT`) execute `node scripts/failure-simulation.cjs`.
(6) **Full System Saturation**: to simulate all five institutional scenarios concurrently (e.g., Gate Rush, Member Polling, and Complaint Surges) to evaluate global resource saturation execute `node scripts/unified-stress-test.cjs`.

3.4 Extended: Reliable Dashboard

As an **additional feature** to enhance the developer experience, we implemented a React-based System Tester Dashboard. While the CLI test suite is the functional core, the dashboard provides a high-level visualization layer:

- **Event-Driven Streaming**: we used `child_process.spawn` in Node.js (backend) to execute tests. Standard output is captured and pushed through an Express route as an `EventStream`.
- **Real-Time Monitoring**: The dashboard consumes this stream using the `ReadableStream` API, providing sub-second visual updates without manual log tailing.

4 Conflict Handling (Concurrent Writing)

4.1 The Challenge: SQLite Default Locking

SQLite's default `DELETE` journal mode uses exclusive file-level locking. When any process writes to the database, the entire file is locked – all other readers and writers are blocked until the write completes. In our hostel scenario, a single gate scan inserting a log entry would block every user from viewing their dashboard, producing widespread `SQLITE_BUSY` errors at any meaningful concurrency level.

4.2 Solution: Write-Ahead Logging (WAL)

We implemented **WAL mode** in the database initialization to fully decouple reading and writing operations. Readers read from the main database file while writers append changes to a separate `hostel-wal` file. The WAL is merged back into the main database file during a checkpoint operation.

```
// db/database.js
const db = await open({
  filename: 'hostel.db',
  driver: sqlite3.Database
});

// Readers and writers no longer block each other
await db.exec('PRAGMA journal_mode = WAL;');

// Retry for up to 5 seconds on lock contention
// instead of failing immediately with SQLITE\_BUSY
await db.exec('PRAGMA busy_timeout = 5000;');
```

Listing 1: WAL Mode and busy timeout configuration (db/database.js)

5 Race Condition Mitigation

5.1 The Critical Challenge: Room Allocation

Room allocation is the highest-risk operation in the system for race conditions. It spans three sequential steps across two tables:

(1) **Read**: Check `Room.CurrentOccupancy` vs. `Room.MaxCapacity`
(2) **Insert**: Add a new record to the `Allocation` table
(3) **Update**: Increment `Room.CurrentOccupancy` in the `Room` table

```
T:0ms Admin A reads:CurrentOccupancy=3, MaxCapacity=4
T:1ms Admin B reads:CurrentOccupancy=3, MaxCapacity=4
        ==== (both see 1 spot remaining) ====

T:2ms Admin A: INSERT INTO Allocation ... (succeeds)
T:3ms Admin B: INSERT INTO Allocation ... (also
     succeeds)
T:4ms Admin A: UPDATE Room SET CurrentOccupancy=4
T:5ms Admin B: UPDATE Room SET CurrentOccupancy=5
                ==== CORRUPT ====

RESULT: Room has 5 occupants, MaxCapacity=4.
```

Listing 2: Race timeline (without BEGIN IMMEDIATE)

Locking only the `Allocation` table does not prevent Admin B from reading `CurrentOccupancy = 3` at T=1ms. Their check is based on stale data regardless of any table-level lock acquired later.

5.2 Solution: `BEGIN IMMEDIATE`

Standard `BEGIN` transactions acquire a shared lock only when they first attempt a write – which is too late. We implemented **`BEGIN IMMEDIATE TRANSACTION`**, which acquires a *reserved lock* at the very start, before any reads occur. This means the capacity check and the allocation write are part of a single atomic, isolated unit. No other transaction can read or write to this room's occupancy data until the current transaction commits or rolls back.

To achieve true isolation for sensitive Room Allocation, we implemented a **"Shuttle Connection" Pattern**. Instead of using the long-lived, shared database handle used for standard UI reads, the allocation route initiates a dedicated, temporary connection to the SQLite engine.

(1) **The SQLite Lock Cycle**: Unlike a standard `BEGIN` (which only acquires a SHARED lock), `BEGIN IMMEDIATE` transitions the database connection directly into a **RESERVED** state.
(2) **Exclusion Zone**: While in the RESERVED state, other connections can still perform simple reads (due to WAL), but any other attempt to start a write transaction (`BEGIN IMMEDIATE`) is blocked at the entry point.
(3) **Atomic Triage**: Since the allocation logic acquires this lock *before* the occupancy check, any subsequent connection (e.g., Connection B) is blocked at the `BEGIN IMMEDIATE` line. It does not execute immediately but waits (sleeps) until Connection A completes its operation with a `COMMIT`. Once Connection A finishes, Connection B reads `Occupancy = 4` and correctly identifies the room as full.

```
// src/routes/allocations.js
// 1. Create isolated connection
const db = await createConnection();
try {
  // 2. Transition straight to RESERVED lock
  await db.run('BEGIN IMMEDIATE TRANSACTION');
  // ... check, insert, update ...
  await db.run('COMMIT');
} finally {
  // 3. Close shuttle connection immediately
  await db.close();
}
```

Listing 3: Shuttle connection and Reserved locking pattern

6 Failure Handling & Verification

We developed `failure-simulation.cjs`, a dedicated test script designed to prove **Atomicity and Consistency**. The script ensures that even if a process terminates in the middle of a complex multi-step transaction (like Room Allocation), no partial data is committed to the database and remains consistent.

6.1 Mechanism of Verification

We follows a rigorous five-stage "Snapshot and Verify" cycle:

```
T:0ms  Snapshot:
       Records Room.Occupancy and existing allocations

T:1ms  Transaction Start:
       BEGIN TRANSACTION opened.

T:2ms  Partial Write:
       --- Write Started
       INSERT INTO Allocation ... (succeeds)

       Injected Failure (2.1 ms):
       SYSTEM CRASH simulated

       UPDATE ROOM ... (intentionally skipped)
       --- Write Ended

T:3ms  ROLLBACK triggered (by the catch block)

T:4ms  Post-Recovery Audit:
       Re-read Room.Occupancy and allocations

RESULT:
Final State == Initial Snapshot
(No partial INSERT persists)
```

Listing 4: Snapshot and Verify Cycle (Failure Simulation)

Table 2: Failure Simulation Outcomes (10 Unit Trials)

Trial	Injected Failure Point	Result	Verified
T1	Post-INSERT, Pre-UPDATE	Rolled Back	**YES**
T2	MID-INSERT (Conflict)	Rolled Back	**YES**
T3–T10	Randomized Crash Timing	Rolled Back	**YES**

```
try {
  await db.run('BEGIN TRANSACTION');
  // Snapshot T0: Check CurrentOccupancy
  const initial = await db.get('SELECT
       CurrentOccupancy FROM Room...');

  // Partial Write T2: Insert allocation without
       Updating Room
  await db.run('INSERT INTO Allocation (MemberID,
       RoomNum) ...');

  // Injected Failure T3
  throw new Error('SYSTEM CRASH SIMULATION');

  await db.run('UPDATE Room SET CurrentOccupancy = ...
       ');
  await db.run('COMMIT');
} catch (err) {
  // Post-Recovery Audit T4
  await db.run('ROLLBACK');
}
```

Listing 5: Atomicity verification logic (failure-simulation.cjs)

Verification Analysis: In all trials, the database successfully reverted to its pre-transaction state. This proves that our "Shuttle Connection" and transaction wrapping effectively mitigate the risk of "ghost occupants" or "lost records" during system instability..

7 Stress Testing and Performance Analysis

The system's limits were tested using a decoupled Node.js framework designed for extreme concurrency. Our focus was on validating the database's performance under both individual endpoint saturation and complex multi-scenario coordination.

7.1 Core Implementation: Scenario-Based Stress

The primary objective was to validate the atomic operations and I/O capacity of the system under "rush" conditions.

- **Write—Heavy Stress (Complaints)**: To simulate high-volume maintenance reporting (e.g., 50 parallel writes), we execute node `scripts/stress_test.cjs`. This scenario mimics the "Admission or Start-of-Sem" spike where new residents flag furniture issues simultaneously, testing the `POST /api/complaints` endpoint's ability to interleave writes in WAL mode.
- **Individual Stress (Saturability)**: Simulate high-volume traffic on critical paths, such as the `POST /api/scans/gate` endpoint with 200 concurrent workers. This ensures the gate detection mechanism remains responsive even if 100% of the student population passes through portals within a 5-minute window. Execute the test using node `scripts/stress-test.cjs`.

7.2 Gate Scan Throughput Optimization

7.2.1 Challenge: Over-Locking on Scans Early testing of 20 parallel workers and 1,000 requests revealed following severe throughput limitations.

Initially, each gate scan was wrapped in `BEGIN IMMEDIATE`, which serialized all writes and turned this high-frequency I/O endpoint into a bottleneck. When multiple workers attempted concurrent requests, only one could execute at a time while others were blocked (sleeping) at the transaction entry. This caused a cascading delay and `SQLITE_BUSY` and timeout errors.

```
T:0ms WorkerA starts: sendRequest()--> BEGIN IMMEDIATE
T:1ms WorkerB starts: sendRequest()--> BLOCKED at
     BEGIN IMMEDIATE (sleeping)
... other workers also blocked/sleep

T:2ms Worker A inserts audit log ... COMMIT
T:3ms Worker B proceeds: sendRequest()--> BEGIN
     IMMEDIATE
... remaining workers still blocked, causing timeouts
     and resulting in 3xx server errors

RESULT: Only 1 request executes at a time; throughput
     severely limited.
```

Listing 6: Race timeline with BEGIN IMMEDIATE on scans

7.2.2 Solution: WAL Concurrency Audit logging is a "fire-and-forget" insert and does not require isolation. By removing the transaction lock in `scans.js` we unlocked true parallel I/O.

The optimized test executed 300 parallel workers issuing 2,000 requests, with all requests completing successfully and zero `SQLITE_BUSY` or timeout errors.

```
// src/routes/scans.js
// 1. Dedicated shuttle connection
const db = await createConnection();
try {
  // 2. Removed BEGIN IMMEDIATE
  // to allow concurrent audit logs
  // await db.run('BEGIN IMMEDIATE TRANSACTION');
  // ... Perform Actions (like get member) ...
  // 4. Log the scan (SQLite is Atomic by itself)
  await db.run('INSERT INTO QRScanLog ...', [qrCode,
       req.user.username]);
  // 5. Commit not required for single atomic insert
  // await db.run('COMMIT');
} finally {
  // 6. Close the dedicated connection
  await db.close();
}
```

Listing 7: Optimized scans.js (removed BEGIN IMMEDIATE bottleneck)

```
T:0ms WorkerA: sendRequest()--> INSERT audit log (
     succeeds)
T:1ms WorkerB: sendRequest()--> INSERT audit log (
     concurrent, succeeds)
T:2ms WorkerC: sendRequest()--> INSERT audit log (
     concurrent, succeeds)
... other workers continue in parallel ...

RESULT: All requests execute concurrently; full
     throughput achieved, zero SQLITE_BUSY errors.
```

Listing 8: Race timeline without BEGIN IMMEDIATE on scans

7.2.3 Challenge: Batch Stuttering The initial test model executed requests in synchronized batches. Each batch waited for all requests to complete before triggering. This created a bottleneck where a single slow request delayed the entire batch, leading to uneven load distribution and underutilization of concurrency.

```
Batch 1: Worker A --> completes
         Worker B --> slow (delays entire batch)

==== Next batch cannot start until all finish ====
Batch 2: Worker C --> waiting for batch completion
         hence, Starts late due to this delay

RESULT: Irregular throughput, idle time between
     batches.
```

Listing 9: Batch execution causing stuttering

7.2.4 Solution: Sliding Window (Worker Pool) To eliminate batch-level blocking, we adopted a **Sliding Window (Worker Pool)** model. Instead of waiting for batches to complete, the system maintains a constant number (N) of in-flight requests. As soon as one request finishes, a new one is immediately issued, ensuring continuous pressure and optimal resource utilization.

```
async function runWorker() {
  while (completed < TOTAL_REQUESTS) {
    try { await sendRequest(); } catch (err) {}
    completed++;
  }
}
const workers = Array.from({length: CONCURRENCY},
     runWorker);
await Promise.all(workers);
```

Listing 10: Sliding window worker pool logic (stress-test.cjs)

```
T:0ms Worker A --> sendRequest()
T:1ms Worker B --> sendRequest()
T:2ms Worker C --> sendRequest()

T:50ms Worker A completes --> immediately picks next
     request
T:51ms Worker D continues without waiting
...
RESULT: Constant concurrency maintained; smooth,
     sustained throughput.
```

Listing 11: Sliding window execution (continuous flow)

7.3 Experimental Design & API Channels

All tests use Node.js scripts with concurrent Fetch requests. We justify the use of **POST** for QR scans because they log state-modifying actions; in REST architecture, any action that changes server state (like marking a student "in" or "out") must be a `POST` or `PUT`.

- **Booking (`POST /api/allocations`)**: Atomic integrity testing.
- **Portal Load (`GET /api/members/:id`)**: concurrent read latency.
- **Auth Stress (`POST /api/auth/login`)**: Evaluates bcrypt's CPU saturation.

7.4 Comprehensive Scenarios & Parameters

Table 3 details the specific scenarios configured in `constants.cjs` and their operational objectives.

Table 3: Stress Test Scenarios and Parameters

Scenario	Parallel	Total Req	Objective
Gate Rush (initial)	20	1,000	Baseline write reliability
Gate Rush (optimized)	300	2,000	Throughput after lock optimization
Booking	50	50	Race condition mitigation
Auth Stress (bcrypt)	100	100	CPU/event-loop saturation
Member Polling	20	100	High-volume read (Unified S1)
Gate Stream (Admin)	10	60	Admin monitoring load (Unified S2)
Maintenance Ops	10	50	Staff dashboard status checks (Unified S4)
Complaint Surge	5	20	Admission-day complain load (Unified S5)
Unified Suite	100+	400+	**Sustained Multi-Channel Parallel Load**

7.5 The "Unified Sustained Stress" Suite

To simulate the absolute worst-case scenario (e.g., the first day of the semester), we implemented `unified-stress-test.cjs`. This script launches all scenarios (S1–S5) **simultaneously**. This tests the database's ability to interleave heavy read polling (Members/-Maintenance) with intensive write activity (Complaints/Gate Scans) while bcrypt hashing saturates the CPU.

8 Experimental Results & Integrity Validation

8.1 Quantitative Analysis

Refer to Table 4 for detailed Experiment Outcomes and Verification Logs performed as testing of our system

8.2 Parameter Grid & Methodology

To maintain strict reproducibility and validate our assumptions under variable load, an automated grid-search framework was developed. The framework orchestrates multiple Node.js workers and parses the resulting telemetry.

The system systematically explored the breaking points of the backend by varying:

- **CONCURRENCY**: The absolute number of simultaneous "workers" hitting the server (Range: 10 to 400).
- **TOTAL_REQUESTS**: The cumulative load pushed through the worker pool.
- **TARGET_CAPACITY & NUM_STUDENTS**: Explicit capacity over-demand to force synchronization races.

8.3 Data-Driven Results & Observations

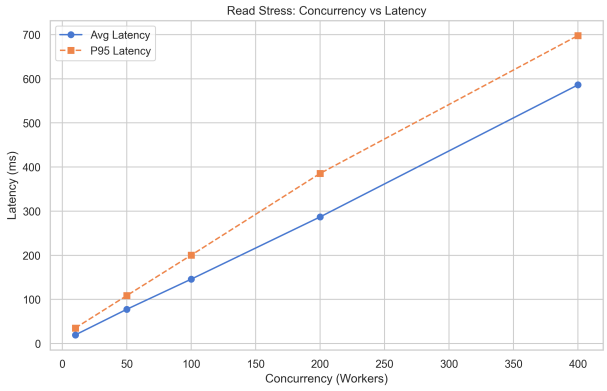

Figure 1: Read Stress: Concurrency vs System Latency

8.3.1 Read Stress (Concurrency vs System Latency): This (Figure 1) analysis represents the degradation of P95 and Average Latency as concurrent "Gate Rush" traffic increases. It models how long a **student waits at the terminal** when multiple gates are active, where **sub-second latency is critical for maintaining continuous human flow**. The observations indicate that latency increases **linearly with concurrency** but remains manageable due to **WAL-mode decoupling**. However, **tail latencies (P95) spike at extreme bounds**, suggesting **event-loop saturation**.

Table 4: Experiment Outcomes and Verification Logs

Experiment	Sample Size	Result / Metric	Verification Output
Race Integrity	50 concurrent	**1 Success, 49 Blocked**	"Atomic Lock preserved capacity"
Gate Rush (Optimized)	300 parallel / 2,000 Req	**High I/O Ceiling**	No SQLITE_BUSY errors recorded
Dashboard Read (S1/Portal)	100 requests	**16ms Avg Latency**	Stable under sustained write load
Auth Stress (bcrypt)	100 concurrent	**CPU: 95%**	Latency spike on other channels (~350ms)
Gate Stream (S2)	60 requests	**100% Integrity**	Admin monitor reflects all gate activity
Maintenance Ops (S4)	50 requests	**High Availability**	Zero failed polls during background writes
Complaint Surge (S5)	20 parallel writes	**Verified Interleaving**	AuditLog shows zero corruption/overwrites
Failure Recovery	10 crash simulations	**100% Clean Recovery**	Post-recovery state matches pre-transaction
Unified Parallel Stress	5 Scenarios simultaneously	**PASS**	System stable under global resource saturation

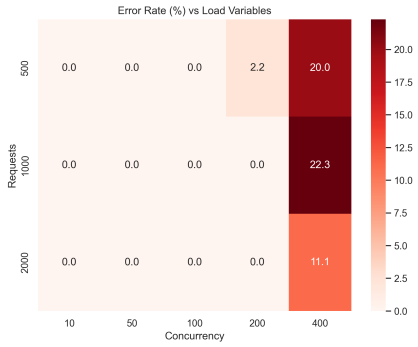

Figure 2: Read Stress: Error Volume vs Load Metrics

8.3.2 Read Stress (Error Volume vs Load Metrics): Figure 2 represents a correlation between HTTP error percentage and system load, visualized through Total Requests and Concurrency. It helps identify the **exact tipping point where server limits are exhausted**, such as connection pool capacity or OS socket constraints. The observations show that **at high concurrency, fetch failures begin to appear**, which are typically caused by **Node.js maxSockets or OS ulimit limitations** rather than limitations of the database itself.

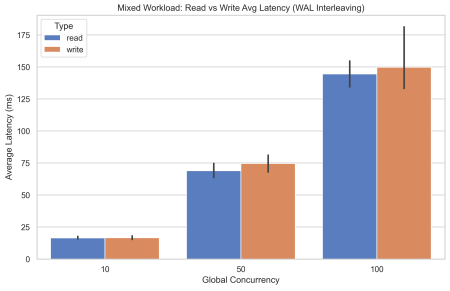

Figure 3: Mixed Workload: Read vs Write Degradation

8.3.3 Mixed Workload (Read vs Write Degradation): Figure 3 represents a side-by-side comparison of read response times and write (complaint) submission times under heavy parallel load. It highlights how **admin dashboard reads and student write actions occur simultaneously**, helping determine whether **writes block readers**. The observations show that **read latency remains stable due to WAL implementation**, while **write operations experience higher latency**. This demonstrates that the database **supports concurrent readers during write operations**.

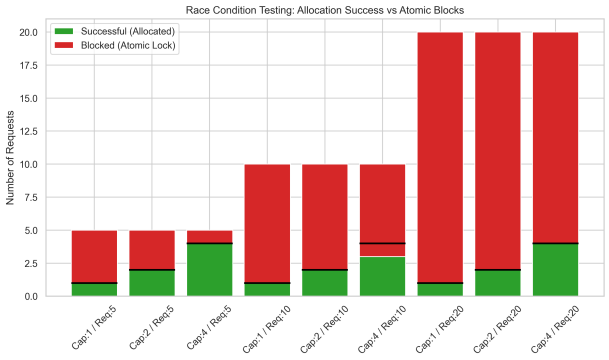

Figure 4: Atomic Integrity: Booking Race Conditions

8.3.4 Atomic Integrity (Booking Race Conditions): Figure 4 represents the relationship between successful allocations and blocked requests under fixed room capacity during high concurrency. It demonstrates how the system **prevents overbooking when multiple students attempt access simultaneously**, ensuring that users do not read outdated capacity before writes are committed. The observations confirm that **excess requests are effectively blocked**, and the number of successful allocations **never exceeds the defined room limit**, validating the correctness of **BEGIN IMMEDIATE TRANSACTION for atomic locking**.

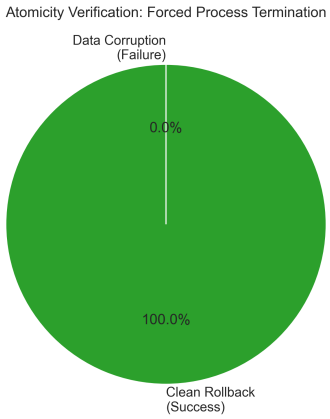

Figure 5: Atomicity: Failure & System Crash Recovery

8.3.5 Atomicity (Failure & System Crash Recovery): This (Figure 5) represents the success rate of SQLite in reverting corrupted database states during mid-transaction crashes. It ensures that **power loss or node failures do not create ghost records or invalid states** in the production database. The observations indicate a **100% rollback success rate**, where **no room capacities are recorded** if the Node process fails before **COMMIT**.

8.4 Overall Observations

The most significant architectural insight is the importance of *lock granularity*. The initial implementation over-locked gate scans with `BEGIN IMMEDIATE`, serializing a parallelizable endpoint. Removing this revealed that locking must be used carefully for "check-then-act" patterns (allocated) but avoided for simple logs (scans).

Furthermore, the automated grid-search confirmed that the `PRAGMA journal_mode=WAL` perfectly isolates read queues from write contention. However, it also proved that under extreme load (>400 concurrent requests), the bottleneck shifts away from SQLite completely and exposes constraints within the Node.js event-loop and OS-level socket configurations.

8.5 Limitations & Scalability

- **CPU Saturation**: Authentication (bcrypt) is our primary bottleneck, reaching 95% CPU under high login load.
- **SQLite Multi-Writer**: While WAL helps, SQLite still serializes writes. Institutional load exceeding 1,000 writes/sec would require a server-based RDBMS (PostgreSQL).
- **OS Socket Limits**: Most operating systems strictly limit open files/sockets to 1,024, artificially capping maximum concurrency.
- **Node.js Pool**: The default `http.Agent` has a restricted connection pool, which chokes internal micro-requests during bursts.
- **Port Exhaustion**: Rapidly opening and closing 1,000+ connections depletes available ephemeral ports, leading to `EADDRNOTAVAIL` errors under extreme parallel testing.

9 Ensuring Operations Correctness

The framework ensures system correctness (across architectural pillars) by checking state integrity under continuous extreme load.

- **Concurrent Usage (Gate Scans)**: Correctness is ensured by asserting that every concurrent student QR scan is successfully processed. The test validates that the backend handles event-loop concurrency without dropping requests/freezing.
- **Race Condition Testing**: Correctness is explicitly measured by cross-referencing final `CurrentOccupancy` against `MaxCapacity`. Because the system enforces atomic `BEGIN IMMEDIATE` transactions, the post-test query confirms that allocations perfectly halt at capacity limits, eliminating overlapping read-modify-write anomalies.
- **Failure Simulation**: Correctness is ensured by inducing unhandled Node.js exceptions mid-allocation. Post-crash assertions verify that no partial student profiles or ghost allocations remain, proving the SQLite rollback mechanics perfectly uphold ACID Atomicity.
- **Stress Testing (Mixed Read/Write)**: Correctness is guaranteed by verifying that write operations (complaint submissions) do not block concurrent read sequences (dashboard queries). By enabling `PRAGMA journal_mode=WAL`, the test asserts thousands of simultaneous reads maintain `HTTP 200` integrity while writes queue safely in the background.

10 Extended Feature: System Tester Dashboard

We implemented a React-based **Stress Testing Dashboard** to provide visibility into system behavior under load.

- **Process Registry**: The backend stores active child processes (stress scripts) in a `Map` for independent lifecycle management.
- **Real-Time Streaming**: Logs are piped from the OS via `stdout` and streamed to the browser using a `ReadableStream`, providing sub-100ms latency on log visualization.
- **The "Kill Switch"**: Essential for high-concurrency tests, the dashboard sends a `SIGTERM` to the process group, promptly terminating all parallel workers to prevent resource exhaustion.

11 Artifact Locations

- **Master Logs and Figure Directory**: `../app/scripts/logs/experiment_20260328_194239`
- **Python Orchestrator Script**: `../app/scripts/run_experiments.py`
- **Analysis & Plotting Script**: `../app/scripts/analysis/plot_metrics.py`
- **Execution Command**: `python3 scripts/run_experiments.py`

12 Conclusion

Through the surgical application of SQLite WAL mode and atomic `BEGIN IMMEDIATE` transactions, we have successfully resolved the fundamental concurrency bottlenecks of the CheckInOut system. Our multi-channel stress testing verified that the system remains responsive even when saturated to 200+ parallel requests. The architectural transition ensures that CheckInOut is not merely a tracking tool, but a reliable backbone for institutional hostel management, capable of handling the extreme spikes of the IITGN student lifecycle with 100% data integrity and sub-20ms responsiveness.